Master Project Implementation Blueprint

A Step-by-Step Engineering Plan to Complete the Unified Campus Assistant & Student Hub

Project: Knowledge Graph-Based Question Answering System for Efficient Information Retrieval
Department: Computer Science & Engineering | **Institution:** Sphoorthy Engineering College
Under the Supervision of: Mrs. Syeda Neha Shireen, Assistant Professor

Executive Summary & Blueprint Scope: This document outlines the definitive step-by-step engineering plan to bring the Knowledge Graph Campus Assistant from its current implementation state (~85% complete) to a fully finished, production-deployed, and academically defended campus platform. It integrates the newly requested **Student Study Hub & Circulars App**, **Multi-Turn Dialogue Context Memory**, **End-to-End API Integration Testing**, **Empirical Performance Benchmarks**, a complete **Review 2 Presentation Deck**, and a comprehensive **Final Project Report**.

1. High-Level Master Phase Matrix

The build plan is partitioned into six clear, sequential phases with specific deliverables:

Phase	Target Component	Core Deliverables & Actions	Impact
Phase 1	**Student Hub & Notes App**	Frontend `/student` portal, StudyMaterial download cards, circulars board, faculty note uploader, mobile PWA manifest.	Replaces WhatsApp & Classroom with unified hub.
Phase 2	**Multi-Turn Dialogue Context**	Session-backed context memory in `/api/chat` and context carry-over in NLP processor.	Natural follow-ups ('who is HOD?' after CSE).
Phase 3	**API Integration Test Suite**	`test_api_integration.py` running automated HTTP client tests on all Flask routes.	Full automated coverage of real user flows.
Phase 4	**Quantitative Benchmarks**	`benchmark_system.py` measuring latency, accuracy, throughput, and memory sync speed.	Empirical validation (<150ms latency).
Phase 5	**Review 2 Presentation**	14-slide academic PowerPoint (`Project_Review_2_Implementation_Presentation.pptx`).	Ready for university Review 2 defense.
Phase 6	**Final Project Report**	Multi-chapter comprehensive thesis PDF (`Comprehensive_Final_Project_Report.pdf`).	Final submission ready for degree award.

2. Phase 1: Full Implementation of the Student Hub & Study App

Rationale: Students currently rely on noisy WhatsApp groups and Google Classroom streams. Phase 1 establishes an integrated academic hub where students access verified notes and circulars directly, both through dedicated web views and conversational chatbot queries.

Step 1.1: Dedicated Student Hub Web View (`templates/student_hub.html`)

Build a responsive web dashboard accessible at `/student` featuring:
• **Academic Study Materials Grid:** Cards organized by Department, Semester (1–8), and Subject Code.
• **One-Click Resource Downloader:** Instant links for lecture PPTs, unit notes, and question banks.
• **Official Digital Notice Board:** Filterable circulars stream from Admin and HODs.
• **Embedded Mobile Assistant:** Persistent floating chat widget for instant queries.

Step 1.2: Faculty Material Upload Portal & API (`/api/upload-material`)

Implement secure POST route in `app.py` restricted to Faculty and Admin roles. Accepts file uploads (PDF, DOCX, PPTX) with title, subject, department, and semester metadata. Saves files to `static/uploads/materials/` with cryptographic UUIDs and registers them in the `StudyMaterial` database.

Step 1.3: Natural Language Chatbot Query Linking for Notes & Notices

Extend `knowledge_graph/nlp_processor.py` with `material_info` and `notice_info` intents. When a student asks *'Can you give me Operating Systems unit 2 notes?'*, the bot performs exact and fuzzy subject matching, queries `StudyMaterial`, and returns immediate direct download buttons in the chat stream.

Step 1.4: Progressive Web App (PWA) Mobile Enablement

Create `static/manifest.json` and `static/sw.js` (Service Worker) with caching protocols. Configures mobile icons, theme colors, and the 'Add to Home Screen' prompt on Android/iOS, enabling students to install the campus assistant as a native app on their phones.

3. Phase 2: Contextual Dialogue & Multi-Turn Memory

Step 2.1 — Session State Persistence: In `app.py` `/api/chat`, initialize a session context dictionary tracking `active_department`, `active_course`, `active_subject`, and `last_intent`.
Step 2.2 — Short-Query Context Inheritance: In `knowledge_graph/nlp_processor.py`, if an inquiry is ≤ 4 tokens and contains relative query markers (e.g., *'who is the HOD?'*, *'what is the fee structure?'*), the engine automatically injects the active context entity, avoiding repetitive prompts.

4. Phase 3: Automated Route Testing & Integration Test Suite

While `test_er_kg.py` verifies internal graph structures (62 assertions), Phase 3 creates `test_api_integration.py` to test live HTTP endpoints using Flask's test client:
• **Portal Route Tests:** GET `/`, GET `/student`, GET `/login`, GET `/register`.
• **API Payload Tests:** POST `/api/chat` (greetings, department info, materials, fees).
• **Data API Tests:** GET `/api/notices`, GET `/api/materials`, POST `/api/upload-material`.
• **Authentication & RBAC Tests:** Enforce login requirements on admin and upload routes.

5. Phase 4: Quantitative Performance Benchmarking Engine

To back up academic claims of high efficiency, we build `benchmark_system.py` to run empirical tests:
• **Latency Benchmarks:** Run 150 diverse test queries across all 18 intent classes and measure execution time in milliseconds.
• **Graph Traversal Speed:** Measure 1-hop, 2-hop, and 3-hop query resolution times in NetworkX.
• **In-Memory Sync Throughput:** Time the complete rebuild and hot-swap pipeline (`build_graph()`).
• **Classification Accuracy:** Calculate precision, recall, and F1-score on benchmark queries.
• **Automated Export:** Saves a structured JSON/console summary ready to be embedded into reports and presentation slides.

6. Phase 5: Project Review 2 Presentation Deck (PPTX)

Build a dedicated PowerPoint presentation script `generate_review_2_presentation.py` producing Project_Review_2_Implementation_Presentation.pptx (14 slides):
1. Title Slide (Review 2 — Implementation & Results, Guide, Team Members)
2. Agenda & Progress Review (Comparison between Review 1 commitments and Review 2 deliveries)
3. System Architecture & Entity-Relationship Knowledge Graph (422 nodes, 22 types, 19 relations)
4. NLP & Fuzzy Matching Pipeline (TF-IDF, Cosine Similarity, Levenshtein fuzzy resolution)
5. Unified Student App & Study Materials Hub (Notes, circulars, PWA mobile integration)
6. Dynamic Admin Synchronization (Hot-reloading without server restarts)
7. Verification & Automated Testing Suite (62 KG tests + API integration tests)
8. Empirical Performance Benchmark Results (sub-150ms latency, throughput tables)
9. System Demo & Screen Flow (Chatbot, Student Hub, Admin Console)
10. Literature Review Gap Resolution (How each 2023–2025 research gap was solved)
11. Roles & Contributions Breakdown (Individual team member accountability)
12. Conclusion & Strategic Roadmap (Neo4j, Local quantized LLM, Timetables)
13. References & Publications Cited
14. Thank You / Q&A; Slide

7. Phase 6: Comprehensive Final Project Report & Thesis (PDF)

Develop an automated report generator `generate_final_project_report.py` compiling a complete, formal, university-grade academic project thesis (`Comprehensive_Final_Project_Report.pdf`):
• **Front Matter:** Official Title Page, Certificate of Originality, Declaration, Acknowledgments, Abstract.
• **Chapter 1: Introduction & Problem Definition:** Academic context, challenges in campus portals, KGQA rationale.
• **Chapter 2: Literature Survey & Gap Analysis:** Comprehensive analysis of 2023–2025 base papers and 5 key gaps.
• **Chapter 3: System Architecture & Knowledge Graph Modeling:** Ontology design, ER schema, entity-relation specifications.
• **Chapter 4: Implementation Methodology:** NLP pipeline, traversal engine, student hub, PWA, dynamic hot-swap sync.
• **Chapter 5: Testing & Performance Benchmarks:** Test matrices, 62+ graph tests, API integration tests, latency benchmarks.
• **Chapter 6: User Interface & Operational Walkthrough:** Portal views, mobile chat UI, student study hub, admin portal.
• **Chapter 7: Conclusion & Future Scope:** Summary of outcomes, institutional benefits, Neo4j & local LLM roadmap.
• **References:** Standard IEEE/ACM citations.

Immediate Execution Sequence: With this plan documented and confirmed, we will now proceed to implement each phase systematically: (1) Student Hub template & routes, (2) Multi-turn chat context, (3) API integration tests, (4) Performance benchmarking script, (5) Review 2 presentation deck, and (6) Comprehensive final project report. All code will be compiled, executed, and verified locally.